Signal Name	Width	Generated In	IF/ID	ID/EX	EX/MEM	MEM/WB	Used In Stage
PC+1	32	IF	✓	✓	✓	-	ID / MEM(JAL)
Instruction	32	IF (IM)	✓	-	-	-	ID
BusA (Rs)	32	ID (RegFile)	-	✓	-	-	EX
BusB (Rt)	32	ID (RegFile)	-	✓	✓	-	EX / MEM(SW)
Extended Imm	32	ID (Ext)	-	✓	-	-	EX
Rs Addr	4	ID	-	✓	-	-	EX (Forward)
Rt Addr	4	ID	-	✓	-	-	EX (Forward)
Rd Addr	4	ID	-	✓	✓	✓	EX / MEM / WB
ALU Result	32	EX (ALU)	-	-	✓	✓	MEM / WB
Mem ReadData	32	MEM (DM)	-	-	-	✓	WB
Equal (EQ)	1	ID (Comp)	-	-	-	-	ID (PC Ctrl)
RegDst	1	ID (CU)	-	✓	-	-	EX
LinkSel	1	ID (CU)	-	✓	✓	✓	WB
ExtOp	1	ID (CU)	-	-	-	-	ID
RegW	1	ID (CU)	-	✓	✓	✓	WB
ALUSrc	1	ID (CU)	-	✓	-	-	EX
MemRd	1	ID (CU)	-	✓	✓	-	MEM
MemWr	1	ID (CU)	-	✓	✓	-	MEM
Wbdata	2	ID (CU)	-	✓	✓	✓	WB
ALUOp	3	ID (CU)	-	✓	-	-	EX
PCSrc	2	ID (CU+PC)	-	-	-	-	ID (PC Ctrl)
Stall	1	Hazard Unit	✓	✓	-	-	IF/ID, ID/EX
Flush	1	Hazard/PC	✓	✓	✓	-	IF/ID to EX/MEM
ForwardA	2	Fwd Unit	-	-	-	-	EX (MUX A)
ForwardB	2	Fwd Unit	-	-	-	-	EX (MUX B)

Instruction	ID Stage Signals	EX Stage Signals	MEM Stage Signals	WB Stage Signals
ADD	RegDst, RegW, ALUOp	RegDst, ALUSrc=0, ALUOp=000	RegW (pass)	RegW=1, Wbdata=10
SUB	RegDst, RegW, ALUOp	RegDst, ALUSrc=0, ALUOp=100	RegW (pass)	RegW=1, Wbdata=10
AND	RegDst, RegW, ALUOp	RegDst, ALUSrc=0, ALUOp=001	RegW (pass)	RegW=1, Wbdata=10
OR	RegDst, RegW, ALUOp	RegDst, ALUSrc=0, ALUOp=010	RegW (pass)	RegW=1, Wbdata=10
XOR	RegDst, RegW, ALUOp	RegDst, ALUSrc=0, ALUOp=011	RegW (pass)	RegW=1, Wbdata=10
NOR	RegDst, RegW, ALUOp	RegDst, ALUSrc=0, ALUOp=101	RegW (pass)	RegW=1, Wbdata=10
SLL	RegDst, RegW, ALUOp	RegDst, ALUSrc=0, ALUOp=110	RegW (pass)	RegW=1, Wbdata=10
SRL	RegDst, RegW, ALUOp	RegDst, ALUSrc=0, ALUOp=111	RegW (pass)	RegW=1, Wbdata=10
CLR	RegDst, RegW	RegDst=1, Wbdata=11	RegW (pass)	RegW=1, Wbdata=11
JR	PCSrc=11	-	-	-
ADDI	RegW, ExtOp, ALUOp	ALUSrc=1, ALUOp=000	RegW (pass)	RegW=1, Wbdata=10
ANDI	RegW, ExtOp=0, ALUOp	ALUSrc=1, ALUOp=001	RegW (pass)	RegW=1, Wbdata=10
ORI	RegW, ExtOp=0, ALUOp	ALUSrc=1, ALUOp=010	RegW (pass)	RegW=1, Wbdata=10
LW	RegW, ExtOp=1, ALUOp	ALUSrc=1, ALUOp=000	MemRd=1	RegW=1, Wbdata=01
SW	ExtOp=1	ALUSrc=1, ALUOp=000	MemWr=1	-

SWAP	RegW, ExtOp=1, ALUOp	ALUSrc=1, ALUOp=000	MemRd=1, MemWr=1	RegW=1, Wbdata=01
BEQ	PCSrc=10 (if EQ=1)	-	-	-
BNE	PCSrc=10 (if EQ=0)	-	-	-
J	PCSrc=01	-	-	-
JAL	PCSrc=01, LinkSel=1	-	RegW (pass)	RegW=1, Wbdata=10, LinkSel=1

Hazard Type	Condition	Action	Affected Signals
EX Forwarding A	EX/MEM.RegW=1 AND EX/MEM.RdAddr!=0 AND EX/MEM.RdAddr=ID/EX.RsAddr	ForwardA = 10	ALU input A = EX/MEM.ALUResult
EX Forwarding B	EX/MEM.RegW=1 AND EX/MEM.RdAddr!=0 AND EX/MEM.RdAddr=ID/EX.RtAddr	ForwardB = 10	ALU input B = EX/MEM.ALUResult
MEM Forwarding A	MEM/WB.RegW=1 AND MEM/WB.RdAddr!=0 AND MEM/WB.RdAddr=ID/EX.RsAddr AND no EX hazard	ForwardA = 01	ALU input A = MEM/WB writeback
MEM Forwarding B	MEM/WB.RegW=1 AND MEM/WB.RdAddr!=0 AND MEM/WB.RdAddr=ID/EX.RtAddr AND no EX hazard	ForwardB = 01	ALU input B = MEM/WB writeback
Load-Use Stall	ID/EX.MemRd=1 AND (ID/EX.RtAddr=IF/ID.RsAddr OR ID/EX.RtAddr=IF/ID.RtAddr)	Stall=1, insert bubble in ID/EX	PC freeze, IF/ID freeze, ID/EX flush
Branch Flush	Branch taken (BEQ: EQ=1) or (BNE: EQ=0)	Flush IF/ID, Flush ID/EX	2 instructions killed (branch penalty=2)
Jump Flush	J or JAL detected in ID stage	Flush IF/ID	1 instruction killed (jump penalty=1)
JR Flush	JR detected in ID stage	Flush IF/ID	1 instruction killed (JR penalty=1)
No Hazard	None of the above conditions	ForwardA=00, ForwardB=00	Normal pipeline operation

Pipeline Signal	Logic Equation / Condition
ForwardA[1]	EX/MEM.RegW AND (EX/MEM.RdAddr ≠ 0) AND (EX/MEM.RdAddr = ID/EX.RsAddr)
ForwardA[0]	MEM/WB.RegW AND (MEM/WB.RdAddr ≠ 0) AND (MEM/WB.RdAddr = ID/EX.RsAddr) AND NOT(ForwardA[1])
ForwardB[1]	EX/MEM.RegW AND (EX/MEM.RdAddr ≠ 0) AND (EX/MEM.RdAddr = ID/EX.RtAddr)
ForwardB[0]	MEM/WB.RegW AND (MEM/WB.RdAddr ≠ 0) AND (MEM/WB.RdAddr = ID/EX.RtAddr) AND NOT(ForwardB[1])
Stall	ID/EX.MemRd AND ((ID/EX.RtAddr = IF/ID.Rs) OR (ID/EX.RtAddr = IF/ID.Rt))
IF/ID.Flush	BranchTaken OR Jump OR JR
ID/EX.Flush	BranchTaken OR Stall
EX/MEM.Flush	BranchTaken (flush late-detected branch)
PC.Write	NOT(Stall) — freeze PC on load-use hazard
IF/ID.Write	NOT(Stall) — freeze IF/ID on load-use hazard

BranchTaken	(Branch AND BEQ AND EQ) OR (Branch AND BNE AND NOT(EQ))
PC MUX Sel	00=PC+1 (default), 01=Jump Target, 10=Branch Target, 11=Rs (JR)